

Memoria Técnica del Proyecto



Descripción del Proyecto

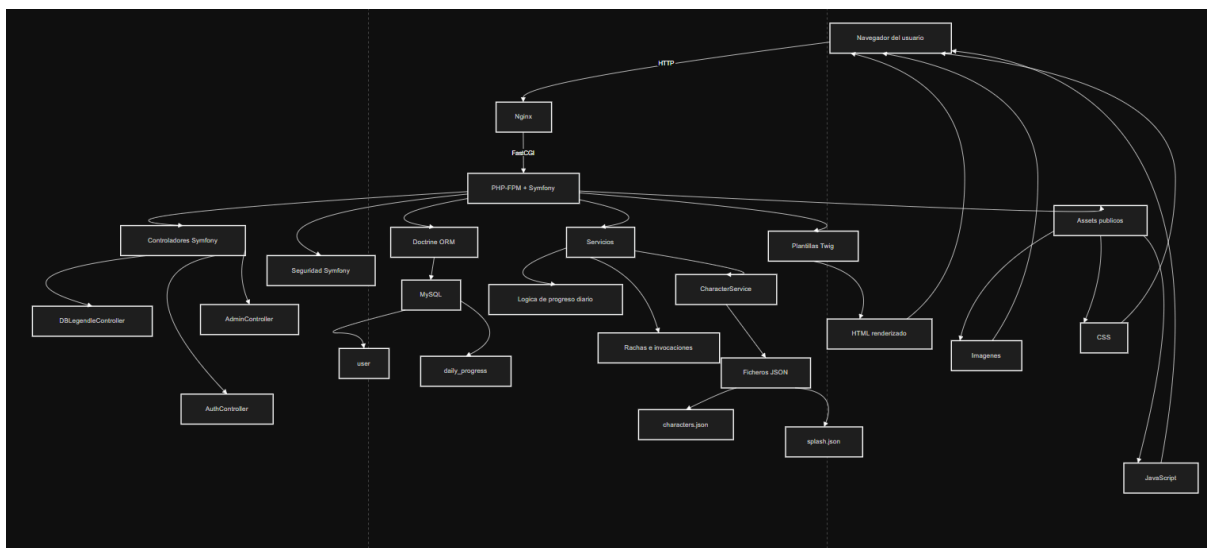
Este proyecto basa sus datos y público al juego gacha de móvil Dragon Ball Legends, surgió la idea en este como consecuencia de la inexistencia de una página o algo del estilo que replique el esquema de juego DLE(mini juegos en los que adivinas personajes en base a coincidencias con otros que introduzcas) por lo que al ser la única página existente con tal comportamiento, sería perfecta para su fandom y podría llamar la atención en este mismo. En si el proyecto se basa en el juego anteriormente descrito, hay un personaje diario que adivinar, insertas personajes y según ciertas coincidencias según atributos, se va adivinando. También a otro modo de juego extra, basado en adivinar personajes no por sus atributos, si no por ciertas partes de sus art cards (splash que muestra la imagen del personaje), con un comportamiento diario igual al anterior modo, además, para ambos existe un modo infinito, por lo que puedes seguir adivinando personajes fuera del desafío diario.

En ambos hay un sistema de racha, que se acumula por el número de veces que se hizo el desafío diario tanto en el clásico como el del art card.

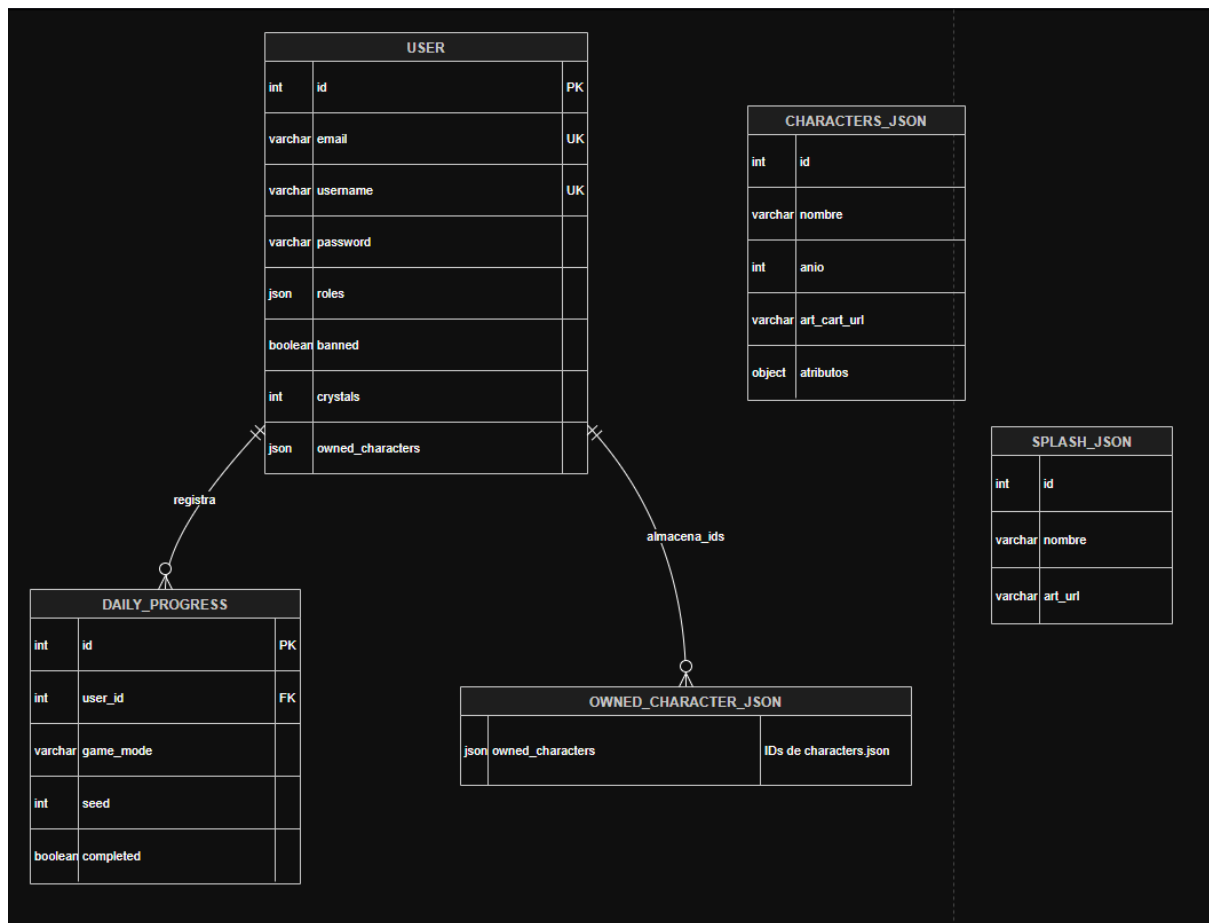
Como tercer apartado principal, existe una página de invocaciones simulando ligeramente al juego original del que se basa, en este se usa como moneda de cambio los cristales, que se ganan haciendo el desafío diario y los del infinito, el objetivo de este es conseguir los más de 680 personajes que hay hasta este momento progresivamente.

Adicionalmente está el apartado de panel de administrador, accesible evidentemente solo por los usuarios con tal rol, donde puedes agregar personajes nuevos, editar los existentes u otros aspectos generales de la web, además de administrar usuarios.

Arquitectura del Sistema



Modelo de datos



El modelo de datos del proyecto se centra en la gestión de usuarios registrados y el seguimiento de su progreso diario dentro de los distintos modos de juego.

La entidad User representa a cada usuario de la aplicación. Almacena sus credenciales, roles de seguridad, estado de baneo, cristales y personajes.

La entidad DailyProgress registra el progreso diario de cada usuario en los modos de juego disponibles actualmente. Cada registro está asociado a un usuario, a un modo de juego concreto y a una fecha. Esto permite controlar si un usuario ya ha completado el reto diario de un modo concreto, calcular las recompensas y obtener las rachas de días consecutivos.

La relación entre User y DailyProgress es de uno a muchos, ya que un usuario puede tener varios registros de progreso, uno por cada día y modo de juego.

El catálogo de personajes no se almacena como tabla relacional, sino como ficheros JSON versionados dentro del proyecto. Esta decisión se debe a que los personajes forman parte de un catálogo estático del juego, utilizado principalmente para consulta, búsqueda y comparación. Estos datos se cargan mediante un servicio de Symfony y pueden modificarse desde el panel de administración en el propio apartado de admins.

Se tuvo que reestructurar el ER al completo ya que el anterior se basaba en una versión del proyecto que en estos momentos considero bastante diferente y obsoleta a como acabó siendo.

Tabla USER

Campo	Tipo	Clave	Descripción
id	int	PK	Identificador único del usuario.
email	varchar	UK	Correo electrónico usado para autenticación.
username	varchar	UK	Nombre público del usuario.
password	varchar	-	Contraseña almacenada mediante hash seguro.
roles	json	-	Lista de roles del usuario, por ejemplo ROLE_USER o ROLE_ADMIN.
banned	boolean	-	Indica si la cuenta está bloqueada.
crystals	int	-	Moneda virtual usada para el sistema de invocación.
owned_characters	json	-	Lista de IDs de personajes obtenidos por el usuario.

Tabla DAILY_PROGRESS

Campo	Tipo	Clave	Descripción
id	int	PK	Identificador único del registro.
user_id	int	FK	Usuario asociado al progreso.
game_mode	varchar	-	Modo de juego: classic o artcart.
seed	int	-	Día del desafío
completed	boolean	-	Indica si el usuario completó ese desafío diario.

Tabla CHARACTERS_JSON

Campo	Tipo	Descripción
id	int	Identificador único del personaje.
nombre	string	Nombre del personaje
anio	int	Año de lanzamiento.
art_cart_url	string	Ruta de la imagen usada en el modo clásico/art cart.
atributos	object	Conjunto de atributos comparables del personaje.

Objeto atributos

Campo	Tipo	Descripción
genero	string	Género del personaje.
afinidad	string	Color o afinidad del personaje.
rareza	string	Rareza: Hero, Extreme, Sparking, LF, Ultra, etc.
estilo	string	Tipo de combate o función.
zenkai	string	Indica si el personaje tiene Zenkai.
saga	string	Saga a la que pertenece.
raza	string	Raza, grupo u ocupación del personaje.

Tabla SPLASH_JSON

Campo	Tipo	Descripción
id	int	Identificador único del splash art.
nombre	string	Nombre del personaje
art_url	string	Ruta de la imagen usada en el modo Art Cart.

Especificaciones Técnicas

Tecnología	Uso en el proyecto	Justificación
Symfony 7	Framework principal del backend.	Arquitectura e integración modular y mantenible.
PHP 8.3	Lenguaje de programación del servidor	Mejor rendimiento y compatibilidad con Symfony.
Twig	Motor de plantillas para renderizar las vistas.	Facilita vistas reutilizables y organizadas.
Doctrine ORM	Gestión de entidades, base de datos y migraciones.	Simplifica el mapeo y persistencia de datos.
MySQL 8	Base de datos relacional	Gestiona información dinámica de forma eficiente.
Nginx	Servidor web del entorno Docker.	Ofrece rapidez y estabilidad en el entorno web.
PHP-FPM	Ejecución de la aplicación Symfony.	Optimiza la ejecución de aplicaciones PHP.
Docker Compose	Orquestación de contenedores.	Permite entornos reproducibles y fáciles de desplegar.
Tailwind CSS	Diseño visual y maquetación responsive.	Facilita interfaces adaptables con clases utilitarias
JavaScript	Interactividad de los modos de juego.	Implementa lógica dinámica en el frontend.
JSON	Almacenamiento del catálogo de personajes y splash arts.	Formato ligero y versionable para datos

Dependencia	Función
symfony/framework-bundle	Núcleo del framework Symfony.
symfony/routing	Gestión de rutas.
symfony/twig-bundle	Integración de Twig.
twig/twig	Motor de plantillas.
symfony/security-bundle	Autenticación y autorización.
symfony/form	Formularios Symfony.
symfony/validator	Validaciones.
doctrine/orm	ORM para base de datos.
doctrine/doctrine-bundle	Integración de Doctrine con Symfony.
doctrine/doctrine-migrations-bundle	Sistema de migraciones.
symfony/asset	Gestión de rutas a assets públicos.
symfony/asset-mapper	Gestión moderna de assets en Symfony.
symfony/dotenv	Carga de variables de entorno.
symfony/monolog-bundle	Sistema de logs.
doctrine/doctrine-fixtures-bundle	Carga de datos de prueba.

Manual de despliegue

El proyecto se levanta desde la carpeta raíz de Proyecto, usando el comando:

```
docker compose up -d --build
```

Para ver el estado de los contenedores:

```
docker compose ps
```

Para ejecutar las migraciones:

```
docker compose exec app php bin/console doctrine:migrations:migrate
```

Para cargar datos de prueba fixture:

```
docker compose exec app php bin/console doctrine:fixtures:load --no-interaction
```